

EDS Lab Assignments

Practical 2

Name: Dhanashri Mamidawr

PRN: 202501040059

2.1.1 List Operations

Problem Statement:

Write a Python program that implements a menu-driven interface for managing a list of integers. The program should have the following menu options:

1. Add
2. Remove
3. Display
4. Quit

The program should repeatedly prompt the user to enter a choice from the menu.

Depending on the choice selected, the program should perform the following actions:

- **Add:** Prompts the user to enter an integer and add it to the integer list. If the input is not a valid integer, display "Invalid input".
- **Remove:** Prompts the user to enter an integer to remove from the list. If the integer is found in the list, remove it; otherwise, display "Element not found". If the list is empty, display "List is empty".
- **Display:** Displays the current list of integers. If the list is empty, display "List is empty".
- **Quit:** Exits the program.
- The program should handle invalid menu choices by displaying "Invalid choice". Ensure that the program continues to prompt the user until they choose to quit (option 4).

Code:

```
l1 = []
while True:
    print("1. Add")
    print("2. Remove")
    print("3. Display")
    print("4. Quit")

    n=int(input("Enter choice: "))
    if n==1:
        add = int(input("Integer: "))
        l1.append(add)
```

```

        print(f"List after adding: {l1}")
    elif n==2:
        if len(l1)==0:
            print("List is empty")
        elif len(l1)!=0:
            remove=int(input("Integer: "))
            if remove not in l1:
                print("Element not found")
            else:
                l1.remove(remove)
            print(f"List after removing: {l1}")
    elif n==3:
        if len(l1)==0:
            print("List is empty")
        else:
            print(l1)
    elif n==4:
        break
    else:
        print("Invalid choice")

```

Output:

The screenshot shows the CodeTANTRA IDE interface. On the left, the '2.1.1. List operations' task is displayed, which asks for a Python program to manage a list of integers with a menu-driven interface. The menu options are: 1. Add, 2. Remove, 3. Display, and 4. Quit. The task also specifies the expected behavior for each option and how to handle invalid inputs.

On the right, the file 'listOps.py' is open, showing the implementation of the program. The code uses a while loop to repeatedly prompt the user for a choice. It handles the 'Add' option by appending a new integer to the list, the 'Remove' option by removing an element if it exists, the 'Display' option by printing the current list, and the 'Quit' option by breaking the loop. It also includes error handling for invalid choices and empty lists.

```

1  # write the code...
2  l1 = []
3  while True:
4      print("1. Add")
5      print("2. Remove")
6      print("3. Display")
7      print("4. Quit")
8      n = int(input("Enter choice: "))
9
10     if n == 1:
11         add = int(input("Integer: "))
12         l1.append(add)
13         print(f"List after adding: {l1}")
14
15     elif n == 2:
16         if len(l1) == 0:
17             print("List is empty")
18         elif len(l1) != 0:
19             remove = int(input("Integer: "))
20             if remove not in l1:
21                 print("Element not found")
22             else:
23                 l1.remove(remove)
24             print(f"List after removing: {l1}")
25
26     elif n == 3:
27         if len(l1) == 0:
28             print("List is empty")
29         else:
30             print(l1)
31
32     elif n == 4:
33         break
34
35     else:
36         print("Invalid choice")
37
38

```

CODETANTRA

HomeLearn Anywhere

20251040059@mitace.ac.inSupportLogout

2.1.1. List operations

Write a Python program that implements a menu-driven interface for managing a list of integers. The program should have the following menu options:
1. Add
2. Remove
3. Display
4. Quit
The program should repeatedly prompt the user to enter a choice from the menu. Depending on the choice selected, the program should perform the following actions:
• **Add**: Prompts the user to enter an integer and add it to the integer list. If the input is not a valid integer, display "Invalid input".
• **Remove**: Prompts the user to enter an integer to remove from the list. If the integer is found in the list, remove it; otherwise, display "Element not found". If the list is empty, display "List is empty".
• **Display**: Displays the current list of integers. If the list is empty, display "List is empty".
• **Quit**: Exits the program.
• The program should handle invalid menu choices by displaying "Invalid choice". Ensure that the program continues to prompt the user until they choose to quit (option 4).

Sample Test Cases

Average time0.038 s38.00 msMaximum time0.046 s46.00 ms2 out of 2 shown test case(s) passed1 out of 1 hidden test case(s) passed

Test case 1

Expected output

Actual output

1. Add
2. Remove
3. Display
4. Quit
Enter choice: 1
Integer: 11
List after adding: [11]
1. Add
2. Remove
3. Display
4. Quit
Enter choice: 1
Integer: 26
List after adding: [11, 26]
1. Add
2. Remove
3. Display
4. Quit
Enter choice: 2
Integer: 26
Element not found
1. Add
2. Remove
3. Display
4. Quit
Enter choice: 2
Integer: 26
List after removing: [26]
1. Add
2. Remove

1. Add
2. Remove
3. Display
4. Quit
Enter choice: 1
Integer: 11
List after adding: [11]
1. Add
2. Remove
3. Display
4. Quit
Enter choice: 1
Integer: 26
List after adding: [11, 26]
1. Add
2. Remove
3. Display
4. Quit
Enter choice: 2
Integer: 26
Element not found
1. Add
2. Remove
3. Display
4. Quit
Enter choice: 2
Integer: 26
List after removing: [26]
1. Add
2. Remove

PrevResetSubmitNext

CODETANTRA

HomeLearn Anywhere

20251040059@mitace.ac.inSupportLogout

2.1.1. List operations

Write a Python program that implements a menu-driven interface for managing a list of integers. The program should have the following menu options:
1. Add
2. Remove
3. Display
4. Quit
The program should repeatedly prompt the user to enter a choice from the menu. Depending on the choice selected, the program should perform the following actions:
• **Add**: Prompts the user to enter an integer and add it to the integer list. If the input is not a valid integer, display "Invalid input".
• **Remove**: Prompts the user to enter an integer to remove from the list. If the integer is found in the list, remove it; otherwise, display "Element not found". If the list is empty, display "List is empty".
• **Display**: Displays the current list of integers. If the list is empty, display "List is empty".
• **Quit**: Exits the program.
• The program should handle invalid menu choices by displaying "Invalid choice". Ensure that the program continues to prompt the user until they choose to quit (option 4).

Sample Test Cases

Average time0.038 s38.00 msMaximum time0.046 s46.00 ms2 out of 2 shown test case(s) passed1 out of 1 hidden test case(s) passed

Test case 2

Expected output

Actual output

1. Add
2. Remove
3. Display
4. Quit
Enter choice: 1
Integer: 1
List after adding: [1]
1. Add
2. Remove
3. Display
4. Quit
Enter choice: 1
Integer: 2
List after adding: [1, 2]
1. Add
2. Remove
3. Display
4. Quit
Enter choice: 1
Integer: 3
List after adding: [1, 2, 3]
1. Add
2. Remove
3. Display
4. Quit
Enter choice: 1
Integer: 4
List after adding: [1, 2, 3, 4]
1. Add
2. Remove

1. Add
2. Remove
3. Display
4. Quit
Enter choice: 1
Integer: 1
List after adding: [1]
1. Add
2. Remove
3. Display
4. Quit
Enter choice: 1
Integer: 2
List after adding: [1, 2]
1. Add
2. Remove
3. Display
4. Quit
Enter choice: 1
Integer: 3
List after adding: [1, 2, 3]
1. Add
2. Remove
3. Display
4. Quit
Enter choice: 1
Integer: 4
List after adding: [1, 2, 3, 4]
1. Add
2. Remove

PrevResetSubmitNext

CODETANTRA

HomeLearn Anywhere

20251040059@mitace.ac.inSupportLogout

2.1.1. List operations

Write a Python program that implements a menu-driven interface for managing a list of integers. The program should have the following menu options:
1. Add
2. Remove
3. Display
4. Quit
The program should repeatedly prompt the user to enter a choice from the menu. Depending on the choice selected, the program should perform the following actions:
• **Add**: Prompts the user to enter an integer and add it to the integer list. If the input is not a valid integer, display "Invalid input".
• **Remove**: Prompts the user to enter an integer to remove from the list. If the integer is found in the list, remove it; otherwise, display "Element not found". If the list is empty, display "List is empty".
• **Display**: Displays the current list of integers. If the list is empty, display "List is empty".
• **Quit**: Exits the program.
• The program should handle invalid menu choices by displaying "Invalid choice". Ensure that the program continues to prompt the user until they choose to quit (option 4).

Sample Test Cases

Average time0.038 s38.00 msMaximum time0.046 s46.00 ms2 out of 2 shown test case(s) passed1 out of 1 hidden test case(s) passed

Test case 3

Expected output

Actual output

4. Quit
Enter choice: 1
Integer: 3
List after adding: [1, 2, 3]
1. Add
2. Remove
3. Display
4. Quit
Enter choice: 1
Integer: 4
List after adding: [1, 2, 3, 4]
1. Add
2. Remove
3. Display
4. Quit
Enter choice: 2
Integer: 2
List after removing: [1, 3, 4]
1. Add
2. Remove
3. Display
4. Quit
Enter choice: 2
Integer: 6
Element not found
1. Add
2. Remove
3. Display
4. Quit
Enter choice: 3
[1, 3, 4]
1. Add
2. Remove

4. Quit
Enter choice: 1
Integer: 3
List after adding: [1, 2, 3]
1. Add
2. Remove
3. Display
4. Quit
Enter choice: 1
Integer: 4
List after adding: [1, 2, 3, 4]
1. Add
2. Remove
3. Display
4. Quit
Enter choice: 2
Integer: 2
List after removing: [1, 3, 4]
1. Add
2. Remove
3. Display
4. Quit
Enter choice: 2
Integer: 6
Element not found
1. Add
2. Remove
3. Display
4. Quit
Enter choice: 3
[1, 3, 4]
1. Add
2. Remove

PrevResetSubmitNext

2.1.2 Dictionary Operations

Problem Statement:

Write a Python program to perform insertion, update, deletion, and traversal operations on a dictionary. An initial dictionary containing 10 predefined records is already given in the program.

Operations to be Performed:

1. Insertion – Insert a new key-value pair into the dictionary using user input.
2. Update – Update the value of an existing key using user input.
3. Deletion – Delete a specified key from the dictionary using user input.
4. Traversal – Traverse the final dictionary and display all key-value pairs.

Input and Output Format:

1. Read an integer representing the key to be inserted and a string representing its value. Insert this new key-value pair into the dictionary and display the dictionary after insertion.
2. Read an integer representing the key to be updated and a string representing the new value. Update the value of the specified key (only if it exists) and display the dictionary after the update.
3. Read an integer representing the key to be deleted. Delete the specified key from the dictionary (only if it exists) and display the dictionary after deletion.
4. Finally, traverse the dictionary and display all key-value pairs.

The program should also display the original dictionary before performing any operations. Each output should be printed with appropriate messages.

Code:

Initial dictionary with 10 predefined records

```
student = {  
    1: "Amit",  
    2: "Riya",  
    3: "Kiran",  
    4: "Neha",  
    5: "Arjun",  
    6: "Pooja",  
    7: "Rahul",  
    8: "Sneha",  
    9: "Vikram",  
    10: "Anjali"  
}
```

```

# write your code here..
print("Original Dictionary:", student)

new_key = int(input())
new_value = input()
student[new_key] = new_value
print("After Insertion:", student)

update_key = int(input())
new_val = input()
if update_key in student:
    student[update_key] = new_val
print("After Update:", student)

del_key = int(input())
if del_key in student:
    del student[del_key]
print("After Deletion:", student)

print("Traversing Dictionary:")
for key, value in student.items():
    print(f"{key} : {value}")

```

Output:

The screenshot displays the CodeTANTRA IDE interface. On the left, a panel titled "2.1.2. Dictionary Operations" provides instructions for a Python program. The main editor shows the code, and the right-hand pane displays the execution output.

Instructions from the IDE:

Write a Python program to perform insertion, update, deletion, and traversal operations on a dictionary. An initial dictionary containing 10 predefined records is already given in the program.

Operations to be Performed:

1. Insertion – Insert a new key-value pair into the dictionary using user input.
2. Update – Update the value of an existing key using user input.
3. Deletion – Delete a specified key from the dictionary using user input.
4. Traversal – Traverse the final dictionary and display all key-value pairs.

Input and Output Format:

1. Read an integer representing the key to be inserted and a string representing its value. Insert this new key-value pair into the dictionary and display the dictionary after insertion.
2. Read an integer representing the key to be updated and a string representing the new value. Update the value of the specified key (only if it exists) and display the dictionary after the update.
3. Read an integer representing the key to be deleted. Delete the specified key from the dictionary (only if it exists) and display the dictionary after deletion.
4. Finally, traverse the dictionary and display all key-value pairs.

The program should also display the original dictionary before performing any operations. Each output should be printed with appropriate messages.

Note:

- All operations must be performed using dictionary methods.
- Perform deletion only if possible; leave the dictionary unchanged.
- Refer to the visible test cases for better understanding and strictly match with the input/outputs.

Sample Test Cases

Execution Output:

```

1 # Initial dictionary with 10 predefined records
2 student = {
3     1: "Amit",
4     2: "Riya",
5     3: "Kiran",
6     4: "Neha",
7     5: "Arjun",
8     6: "Pooja",
9     7: "Rahul",
10    8: "Sneha",
11    9: "Vikram",
12    10: "Anjali"
13 }
14
15 # write your code here...
16 print("Original Dictionary:", student)
17
18 # Insertion
19 insert_key = int(input())
20 insert_value = input()
21 student[insert_key] = insert_value
22 print("After Insertion:", student)
23
24 # Update
25 update_key = int(input())
26 update_value = input()
27 if update_key in student:
28     student[update_key] = update_value
29 print("After Update:", student)
30
31 # Deletion
32 del_key = int(input())
33 if del_key in student:
34     student.pop(del_key)
35 print("After Deletion:", student)
36
37 print("Traversing Dictionary:")
38 for key, value in student.items():
39     print(f"{key} : {value}")

```

CODETANTRA

HomeLearn Anywhere

20201040209@mitace.ac.inSupportLogout

2.1.2. Dictionary Operations

Write a Python program to perform insertion, update, deletion, and traversal operations on a dictionary. An initial dictionary containing 10 predefined records is already given in the program.

Operations to be Performed:

1. Insertion – Insert a new key-value pair into the dictionary using user input.

2. Update – Update the value of an existing key using user input.

3. Deletion – Delete a specified key from the dictionary using user input.

4. Traversal – Traverse the final dictionary and display all key-value pairs.

Input and Output Format:

1. Read an integer representing the key to be inserted and a string representing its value. Insert this new key-value pair into the dictionary and display the dictionary after insertion.

2. Read an integer representing the key to be updated and a string representing the new value. Update the value of the specified key (only if it exists) and display the dictionary after the update.

3. Read an integer representing the key to be deleted. Delete the specified key from the dictionary (only if it exists) and display the dictionary after deletion.

4. Finally, traverse the dictionary and display all key-value pairs.

The program should also display the original dictionary before performing any operations. Each output should be printed with appropriate messages.

Note:

All operations must be performed using dictionary methods.

Perform deletion only if possible; leave the dictionary unchanged.

Refer to the visible test cases for better understanding and strictly match with the input/outputs.

Sample Test Cases

Engineer

Average time0.009 sMaximum time0.011 s

2 out of 2 shown test case(s) passed2 out of 2 hidden test case(s) passed

Test case 1

Expected output

Original Dictionary: {1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Priya', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 11: 'Suresh'}

11

Suresh

After Insertion: {1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Priya', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 11: 'Suresh'}

12

Kartik

After update: {1: 'Amit', 2: 'Riya', 3: 'Kartik', 4: 'Neha', 5: 'Arjun', 6: 'Priya', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 11: 'Suresh'}

13

After deletion: {1: 'Amit', 2: 'Riya', 3: 'Kartik', 4: 'Neha', 5: 'Arjun', 6: 'Priya', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 11: 'Suresh'}

Traversing Dictionary:

1 : Amit

2 : Riya

3 : Kartik

4 : Neha

5 : Arjun

6 : Priya

7 : Rahul

8 : Sneha

9 : Vikram

10 : Anjali

11 : Suresh

Actual output

Original Dictionary: {1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Priya', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 11: 'Suresh'}

11

Suresh

After Insertion: {1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Priya', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 11: 'Suresh'}

12

Kartik

After update: {1: 'Amit', 2: 'Riya', 3: 'Kartik', 4: 'Neha', 5: 'Arjun', 6: 'Priya', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 11: 'Suresh'}

13

After deletion: {1: 'Amit', 2: 'Riya', 3: 'Kartik', 4: 'Neha', 5: 'Arjun', 6: 'Priya', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 11: 'Suresh'}

Traversing Dictionary:

1 : Amit

2 : Riya

3 : Kartik

4 : Neha

5 : Arjun

6 : Priya

7 : Rahul

8 : Sneha

9 : Vikram

10 : Anjali

11 : Suresh

Test case 2

CODETANTRA

HomeLearn Anywhere

20201040209@mitace.ac.inSupportLogout

2.1.2. Dictionary Operations

Write a Python program to perform insertion, update, deletion, and traversal operations on a dictionary. An initial dictionary containing 10 predefined records is already given in the program.

Operations to be Performed:

1. Insertion – Insert a new key-value pair into the dictionary using user input.

2. Update – Update the value of an existing key using user input.

3. Deletion – Delete a specified key from the dictionary using user input.

4. Traversal – Traverse the final dictionary and display all key-value pairs.

Input and Output Format:

1. Read an integer representing the key to be inserted and a string representing its value. Insert this new key-value pair into the dictionary and display the dictionary after insertion.

2. Read an integer representing the key to be updated and a string representing the new value. Update the value of the specified key (only if it exists) and display the dictionary after the update.

3. Read an integer representing the key to be deleted. Delete the specified key from the dictionary (only if it exists) and display the dictionary after deletion.

4. Finally, traverse the dictionary and display all key-value pairs.

The program should also display the original dictionary before performing any operations. Each output should be printed with appropriate messages.

Note:

All operations must be performed using dictionary methods.

Perform deletion only if possible; leave the dictionary unchanged.

Refer to the visible test cases for better understanding and strictly match with the input/outputs.

Sample Test Cases

Engineer

Average time0.009 sMaximum time0.011 s

2 out of 2 shown test case(s) passed2 out of 2 hidden test case(s) passed

Test case 1

Expected output

Original Dictionary: {1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Priya', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 11: 'Suresh'}

11

Suresh

After Insertion: {1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Priya', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 11: 'Suresh'}

12

Kartik

After update: {1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Priya', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 11: 'Suresh'}

13

After deletion: {1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Priya', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 11: 'Suresh'}

Traversing Dictionary:

1 : Amit

2 : Riya

3 : Kiran

4 : Neha

5 : Arjun

6 : Priya

7 : Rahul

8 : Sneha

9 : Vikram

10 : Anjali

11 : Suresh

Actual output

Original Dictionary: {1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Priya', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 11: 'Suresh'}

11

Suresh

After Insertion: {1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Priya', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 11: 'Suresh'}

12

Kartik

After update: {1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Priya', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 11: 'Suresh'}

13

After deletion: {1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Priya', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 11: 'Suresh'}

Traversing Dictionary:

1 : Riya

2 : Kiran

3 : Kiran

4 : Neha

5 : Arjun

6 : Priya

7 : Rahul

8 : Sneha

9 : Vikram

10 : Anjali

11 : Neha

Test case 2

2.2.1 Linear Search Technique

Problem Statement:

Write a program to check whether the given element is present or not in the array of elements using linear search.

Input format:

- The first line of input contains the array of integers which are separated by space
- The last line of input contains the key element to be searched

Output format:

- If the element is found, print the index.
- If the element is not found, print **Not found**.

Code:

```
def linear_search(arr, key):  
    for i in range(len(arr)):  
        if arr[i] == key:  
            return i  
    return -1
```

```
arr = list(map(int,input().split()))
```

```
key = int(input())
```

```
result = linear_search(arr, key)
```

```
if result != -1:
```

```
    print(result)
```

```
else:
```

```
    print("Not found")
```

Output:

The screenshot displays the CodeTANTRA IDE interface. On the left, the problem statement for '2.2.1 Linear search Technique' is visible, including the input/output format and sample test cases. The main editor shows the Python code for the linear search function and its application. The right sidebar shows the test results, indicating that 2 out of 2 shown test cases passed and 2 out of 2 hidden test cases passed. The test cases are as follows:

Test Case	Expected Output	Actual Output
Test case 1	1 2 3 4 5 6 3	1 2 3 4 5 6 3
Test case 2	1 2 3 4 5 6 7 8 9 Not found	1 2 3 4 5 6 7 8 9 Not found

2.2.2 Captain of the Team

Problem Statement:

You are provided with the heights of 11 cricket players (in centimeters). Your task is to identify the tallest player, who will be selected as the captain of the team.

Input Format:

The first line of input will contain 11 integers, each representing the height of a player (in centimeters), each separated by a space.

Output Format

The output should be the height (in centimeters) of the tallest player.

Code:

```
heights = list(map(int,input().split()))
tallest_player = max(heights)
print(tallest_player)
```

Output:

The screenshot displays the CodeTANTRA web interface. On the left, the problem statement for '2.2.2. Captain of the Team' is visible, including the input and output formats. The main editor shows the Python code: `heights = list(map(int,input().split()))`, `tallest_player = max(heights)`, and `print(tallest_player)`. Below the code, the test results are shown: '1 out of 1 shown test case(s) passed' and '2 out of 2 hidden test case(s) passed'. The test case details for 'Test case 1' show the expected output as `171 169 165 156 174 151 166 158 167 172 168` and the actual output as `171`. The interface also includes a terminal, a debugger, and navigation buttons at the bottom.